Low-Level Architecture

The following diagram describes the [Application 1] architecture:

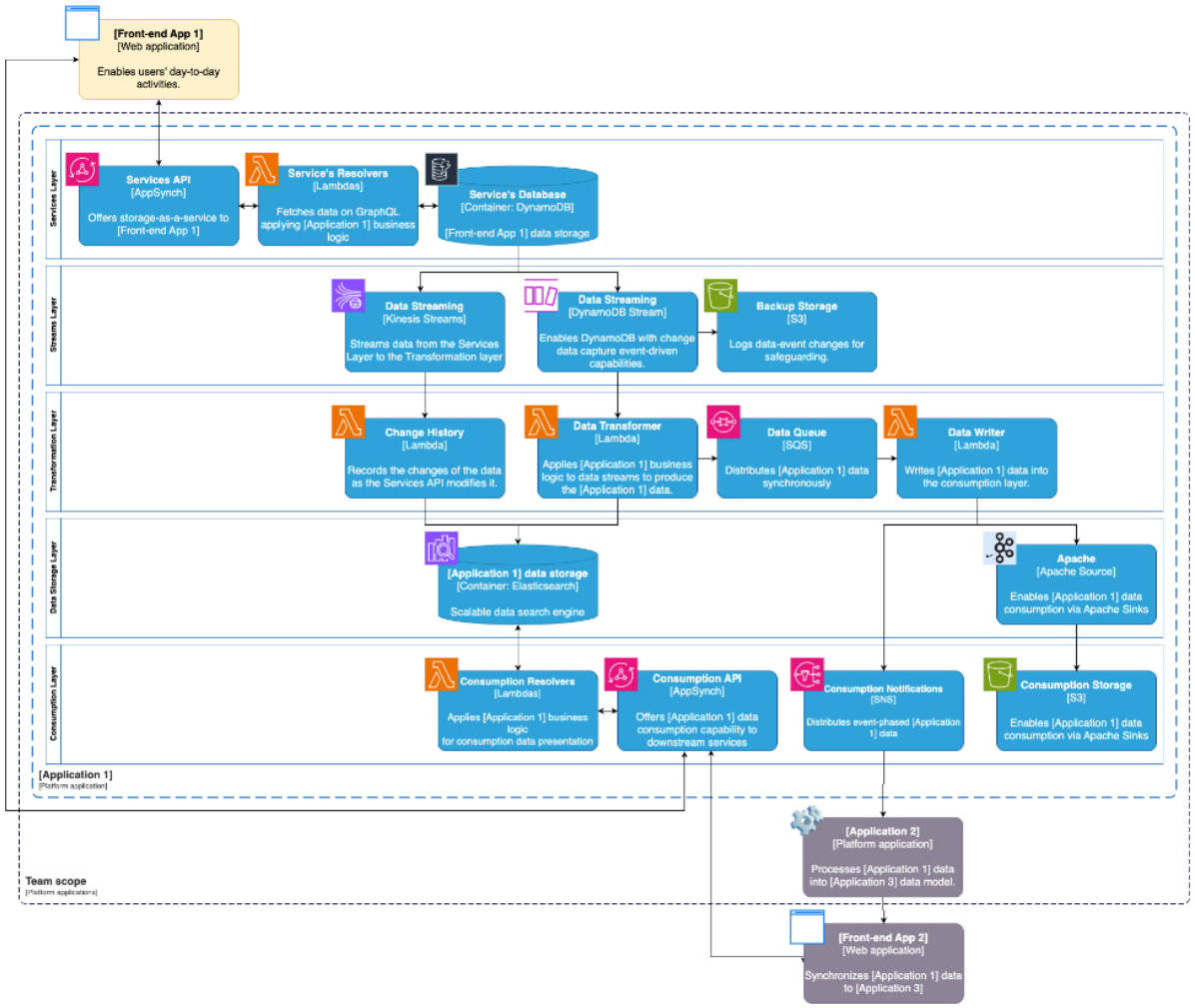

Figure 1. [Application 1] Architecture

The following table lists the architectural elements in alphabetical order, providing additional context of the relationship between them:

Element	Description
Backup Storage	Stores logs of the changes in the [Front-end App 1] data of the **Service's Database**.
Change History	Writes a data history log in the **app data Storage** each time the data within the **Service's Database** changes.
Consumption API	Enables [Front-end App 1] and [Front-end App 2] to consume app data via GraphQL API.
Consumption Notifications	Enables [Application 2] to consume app data via publish and subscribe messaging paradigm.
Consumption Resolvers	Fetches the app data in response to GraphQL querries.
Consumption Storage	Stores the app data from **Apache** and exposes it to Apache Sinks.
Data Queue	Receives and persists app data for the **Consumption Notifications** service.
Data Streaming (DynamoDB Stream)	Distributes the **Service's Database** data to the **Data Transformer** and records the Change Data Capture events in the **Backup Storage** service.
Data Streaming (Kinesis Streams)	Stream data service for the Apache feature (under development)
Data Transformer	Transforms the [Front-end App 1] data applying [Application 1] business logic to produce the final

	app data. Upon completion, the **Data Transformer** stores the app data in the **app data storage** and distributes the app data to the **Data Queue**.
Data Writer	Publishes the **Data Queue**'s app data in the **Consumption Notification** service.
Apache	Stores prepared app data for Apache sinks inside the **Consumption Storage**.
app data storage	Stores the app data.
Service's Database	Stores the [Front-end App 1] data.
Service's Resolvers	Fetches the [Front-end App 1] data in response to GraphQL querries.
Services API	Enables [Front-end App 1] with storage-as-a-service via GraphQL API.

Dataflow

The following steps describe a sequencial explanation of the data flow in the [Application 1] architecture:

1. The **[Front-end App 1]** application leverages the **Services API** to utilize the **Service's Database**.
2. The **Service's Database** distributes to data across the **Kinesis and DynamoDB Data Streaming** services.

   ***The DynamoDB Stream path**:*

   a. The **DynamoDB Data Streaming** service distributes the **Service's Database** data to the **Data Transformer** lambda while also recording the Change Data Capture events in the **Backup Storage**.

   b. The **Data Transformer** lambda transforms the data applying [Application 1] business logic to produce the final app data. The **Data Transformer** lambda then stores the app data in the **app data storage** and distributes it to the **Data Queue**.

   c. The **Data Queue** persists the app data until the **Data Writer** lambda publishes the app data in the **Consumption Notification** and **Apache** services.

   - The **Consumption Notification** service publishes the app data to the [Application 2] services.
   - The **Apache** service publishes the app data inside the **Consumption Storage** that is available for Apache Sinks.

   The Kinesis Streams path:

   a. The **Kinesis Data Streaming** service distributes the **Service's Database** data to the **Change History** lambda.

   b. The **Change History** lambda triggers on every Services API operation to compare the change data between the **Service's Database** and the **app data storage**, updating the history log with change data whenever the Services API modifies the Service's Database data.
3. Users of the **Consumption API** interact with AWS AppSynch, which leverages the **Consumption Resolver** lambdas to request data from the **app data storage**.